

5장. 자료구조와 복잡도 요약 가이드

비전공자도 한눈에 이해하기 쉽게 비유로 풀어쓴 개념 정리 기본서

이 문서는 프로그램의 기본 뼈대가 되는 **자료구조(Data Structure)**와 코드의 효율성을 측정하는 **복잡도(Complexity)**의 핵심 개념을 누구나 이해하기 쉬운 일상적인 비유를 들어 설명합니다.

5.1 복잡도 (Complexity)

작성한 코드가 "얼마나 효율적이고 똑똑하게 작동하는가"를 측정하는 객체적인 기준입니다.

개념	핵심 설명	직관적 이해
시간 복잡도 (Time Complexity)	프로그램이 실행될 때 완료되기까지 얼마나 많은 연산(시간) 이 걸리는가	동일한 작업을 할 때 내비게이션이 얼마나 빠른 길(최단 시간)을 찾아주는지의 개념
공간 복잡도 (Space Complexity)	프로그램이 실행되는 동안 컴퓨터의 메모리(저장 공간) 를 얼마나 차지하는가	방 정리를 할 때 짐을 보관하기 위해 수납 상자를 얼마나 많이 사용하는지의 개념
자료구조에서의 시간 복잡도	특정 자료구조를 사용할 때 데이터를 추가, 삭제, 검색하는 데 걸리는 시간 비교	상자 종류에 따라 물건을 넣고 찾을 때 걸리는 시간이 각각 다를 수 분석

5.2 선형 자료구조 (Linear Data Structure)

데이터를 일렬로, 즉 직선 모양으로 나란히 정렬하여 관리하는 가장 기초적인 방식입니다.

5.2.1 연결 리스트 (Linked List)

데이터들이 한 자리에 모여있지 않고, 각 데이터가 다음 데이터가 있는 위치(주소)를 가리키며 줄줄이 연결된 구조입니다.

일상 비유: 기차 객차 연결 또는 보물찾기 쪽지

중간에 새로운 기차 칸을 끼워 넣거나 뺄 때, 앞뒤 연결 고리만 바꾸면 되므로 데이터 삽입과 삭제가 매우 빠르고 편합니다. 다만, 특정 데이터를 찾으려면 첫 칸부터 차례대로 다음 칸을 확인하며 이동해야 합니다.

5.2.2 배열 (Array)

처음부터 메모리 상에 방 크기를 딱 정해놓고 순서대로 데이터를 나란히 채워 넣는 구조입니다.

일상 비유: 번호가 명확히 지정된 아파트 우편함

각 칸마다 고유한 번호(인덱스)가 붙어 있어, 몇 번째 칸인지만 알면 주소를 통해 빛의 속도로 바로 찾아갈 수 있습니다. 하지만 중간에 방을 더 늘리거나 크기를 조절하기가 매우 까다롭습니다.

5.2.3 벡터 (Vector)

배열의 치명적인 단점인 '고정된 크기'를 보완하여, 공간이 부족해지면 자동으로 크기가 늘어나는 스마트한 배열입니다.

일상 비유: 칸이 자동으로 확장되는 가변형 서랍장

배열처럼 인덱스 번호로 빠른 조회가 가능하면서도, 데이터가 가득 차면 컴퓨터가 알아서 여유 공간을 더 넓혀 주기 때문에 다루기가 매우 편리합니다. (Python의 일반적인 '리스트'가 이와 유사하게 작동합니다.)

5.2.4 스택 (Stack)

한쪽 끝이 막혀 있어 위로만 데이터를 쌓고 위에서부터 꺼낼 수 있는 **후입선출(LIFO - Last In, First Out)** 구조입니다.

일상 비유: 프링글스 과자 통 또는 세로로 쌓인 접시

가장 최근에 넣은 것을 가장 먼저 꺼내야 합니다. 우리가 컴퓨터를 할 때 가장 자주 쓰는 '**되돌리기(Ctrl + Z)**' 기능이나 웹 브라우저의 '**뒤로 가기**'가 이 원리로 움직입니다.

5.2.5 큐 (Queue)

양방향이 뚫려 있어 먼저 들어간 데이터가 먼저 나오는 **선입선출(FIFO - First In, First Out)** 구조입니다.

일상 비유: 맛집 줄 서기 또는 고속도로 톨게이트

가장 먼저 와서 줄을 선 데이터가 가장 먼저 처리되어 나갑니다. 은행의 **번호표 대기 시스템**, 문서 인쇄를 누른 순서대로 처리하는 **프린터 인쇄 대기열** 등에 필수적으로 사용됩니다.